

Web System Technologies

IT0049



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Module 5

Validation

Validation : Overview

Objectives

- To understand the importance of validation as part of security.
- To differentiate server-side validation and client-side validations.
- To explore different validations and error feedback techniques, methods and approaches.
- To create a custom validation.
- To define different validation rule.
- To prep the data before processing.



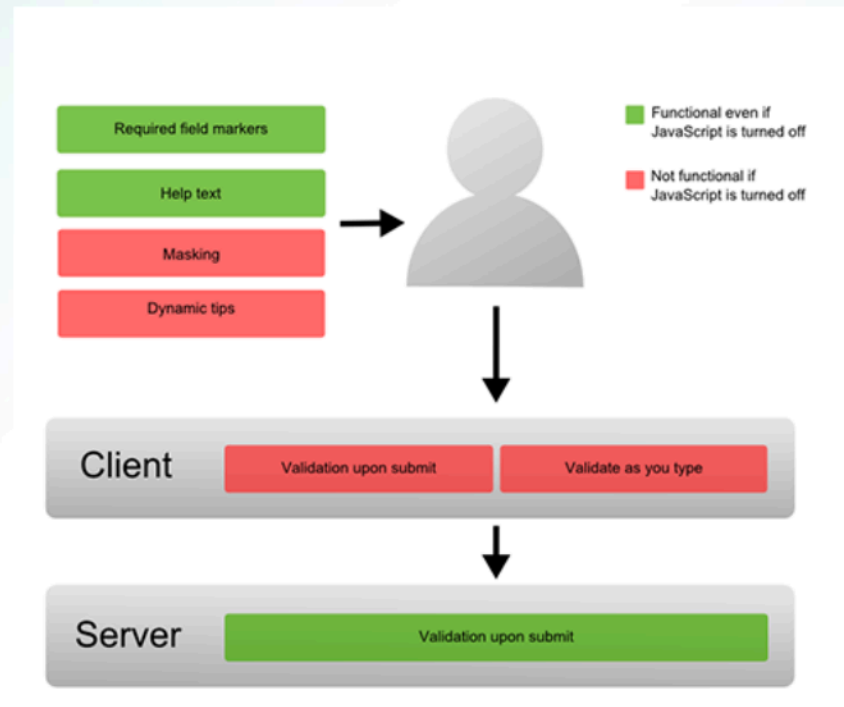
Form Validation

In web applications, users fill out the web form with information to finish tasks successfully. However, human errors are inevitable, this is where **form validation** comes out useful.

Form validation ensures that end-users provided necessary and properly formatted information to successfully complete a transaction.

Validation Methods

User's input can be validated on the server and on client by server-side and client-side validations.



Server-side Validation

- Form data is being sent to the server and validated.
- If the validation fails, the response is then sent back to the client.
- Secure because validations will work even if JavaScript is turned off in web browser.
- Users need to fill out the web form without getting feedback until they submit.
- Results in a slow response from the server.



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Client-side Validation

- Done on the client using script languages such as JavaScript
- User's input can be validated as they type.
- Visually rich validation.
- Form never gets submitted if validation fails.
- It relies on JavaScript. If users turn JavaScript off, they can easily bypass the validation.



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

What to validate?

- **Required fields**
- **Correct Format**
- **Confirmation Fields**



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

CI's Form Validation Library

To simulate CodeIgniter's approach to data validation, here's an ideal scenario

1. A form is displayed.
2. User fill it in and submit.
3. If you submitted something invalid, or missed a required item, the form is redisplayed containing your data along with an error message.
4. This process continues until you have submitted a valid form.



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

CI's Form Validation Library

On the receiving end, the script must:

1. Check for required data.
2. Verify that the data is of the correct type, and meets the correct criteria.
3. Sanitize the data for security.
4. Pre-format the data if needed.
5. Prep the data for insertion in the database.

Form validation, while simple to create, is generally very messy and tedious to implement.



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Form Validation Tutorial

In order to implement form validation you'll need three things:

1. A View file containing a form.
2. A View file containing a “success” message to be displayed upon successful submission.
3. A controller method to receive and process the submitted data.



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

The Form

Create a form called **signup.php**, write this code and save it to your **app/Views/** folder:

```
<html>
<head>
  <title>My Form</title>
</head>
<body>

  <?= validation_list_errors() ?>

  <?= form_open('form') ?>

    <h5>Username</h5>
    <input type="text" name="username" value="<?= set_value('username') ?>" size="50">

    <h5>Password</h5>
    <input type="text" name="password" value="<?= set_value('password') ?>" size="50">

    <h5>Password Confirm</h5>
    <input type="text" name="passconf" value="<?= set_value('passconf') ?>" size="50">

    <h5>Email Address</h5>
    <input type="text" name="email" value="<?= set_value('email') ?>" size="50">

    <div><input type="submit" value="Submit"></div>

  <?= form_close() ?>

</body>
</html>
```



The Success Page

Create a form called **success.php**. In it, place this code and save it to your **app/Views/** folder:

```
<html>
<head>
  <title>My Form</title>
</head>
<body>

  <h3>Your form was successfully submitted!</h3>

  <p><?= anchor('form', 'Try it again!') ?></p>

</body>
</html>
```



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

The Controller

Create a controller called **Form.php**.
In it, place this code and save it to
your **app/Controllers/** folder:

```
<?php

namespace App\Controllers;

class Form extends BaseController
{
    protected $helpers = ['form'];

    public function index()
    {
        if (! $this->request->is('post')) {
            return view('signup');
        }

        $rules = [
            // @TODO
        ];

        $data = $this->request->getPost(array_keys($rules));

        if (! $this->validateData($data, $rules)) {
            return view('signup');
        }

        // If you want to get the validated data.
        $validData = $this->validator->getValidated();

        return view('success');
    }
}
```



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

The Routes

Then add routes for the controller in **app/Config/Routes.php**:

```
// ...  
  
$routes->get('form', 'Form::index');  
$routes->post('form', 'Form::index');  
  
// ...
```



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Try It!

To try your form, visit your site using a URL similar to this one:

`example.com/index.php/form/`

Since you haven't told the `validateData()` method to validate anything yet, it returns `false` (boolean false) by default. The `validateData()` method only returns `true` if it has successfully applied your rules without any of them failing.



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Add Validation Rules

Add validation rules in the controller (**Form.php**):

```
// ...  
  
$rules = [  
    'username' => 'required|max_length[30]',  
    'password' => 'required|max_length[255]|min_length[10]',  
    'passconf' => 'required|max_length[255]|matches[password]',  
    'email'    => 'required|max_length[254]|valid_email',  
];  
  
// ...
```

If you submit the form you should see the success page or the form with error messages.

\$this->validateData()

To simplify data checking, the controller provides the convenience method **validateData()**.

The method accepts (1) an array of data to validate, (2) an array of rules, (3) an optional array of custom error messages to display if the items are not valid, (4) an optional database group to use.

```
<?php
namespace App\Controllers;

class Form extends BaseController
{
    protected $helpers = ['form'];

    public function index()
    {
        if (! $this->request->is('post')) {
            return view('signup');
        }

        $rules = [
            'username' => 'required|max_length[30]',
            'password' => 'required|max_length[255]|min_length[10]',
            'passconf' => 'required|max_length[255]|matches[password]',
            'email' => 'required|max_length[254]|valid_email',
        ];

        $data = $this->request->getPost(array_keys($rules));

        if (! $this->validateData($data, $rules)) {
            return view('signup');
        }

        // If you want to get the validated data.
        $validData = $this->validator->getValidated();

        return view('success');
    }
}
```



Validation Class



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Loading the library

The library is loaded as a service named **validation**:

```
$validation = service('validation');
```

This automatically loads the **Config\Validation** file which contains settings for including multiple Rulesets, and collections of rules that can be easily reused.



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

How validation works

- The validation never changes data to be validated.
- The validation checks each field in turn according to the Validation Rules you set. If any rule returns false, the check for that field ends there.
- The Format Rules do not permit empty string. If you want to permit empty string, add the `permit_empty` rule.
- If a field does not exist in the data to be validated, the value is interpreted as null. If you want to check that the field exists, add the `field_exists` rule.



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Setting Validation Rules

CodeIgniter lets you set as many validation rules as you need for a given field, cascading them in order.

Setting a Single Rule

setRule()

This method sets a single rule. It has the method signature:

```
setRule(string $field, ?string $label, array|string $rules[, array $errors = []])
```

The \$rules either takes in a pipe-delimited list of rules or an array collection of rules:

```
$validation->setRule('username', 'Username', 'required|max_length[30]|min_length[3]');  
$validation->setRule('password', 'Password', ['required', 'max_length[255]', 'min_length[8]', 'alpha_numeric_punct']);
```

The value you pass to \$field must match the key of any data array that is sent in. If the data is taken directly from \$_POST, then it must be an exact match for the form input name.



Setting Validation Rules

Setting Multiple Rules

setRules()

Like setRule(), but accepts an array of field names and their rules:

```
$validation->setRules([
    'username' => 'required|max_length[30]',
    'password' => 'required|max_length[255]|min_length[10]',
]);
// or
$validation->setRules([
    'username' => ['required', 'max_length[30]'],
    'password' => ['required', 'max_length[255]', 'min_length[10]'],
]);
```



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Working with Validation

Running Validation

The **run()** method runs validation. It has the method signature:

```
run(?array $data = null, ?string $group = null, ?string $dbGroup = null): bool
```

The **\$data** is an array of data to validate. The optional second parameter **\$group** is the predefined group of rules to apply. The optional third parameter **\$dbGroup** is the database group to use.

This method returns true if the validation is successful.

```
if (! $validation->run($data)) {  
    // handle validation errors  
}  
// or  
if (! $validation->run($data, 'signup')) {  
    // handle validation errors  
}
```



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Getting Validated Data

The actual validated data can be retrieved with the **getValidated()** method. This method returns an array of only those elements that have been validated by the validation rules.

```
// In Controller.

if (! $this->validateData($data, [
    'username' => 'required',
    'password' => 'required|min_length[10]',
])) {
    // The validation failed.
    return view('login', [
        'errors' => $this->validator->getErrors(),
    ]);
}

// The validation was successful.

// Get the validated data.
$validData = $this->validator->getValidated();
```



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

How to save your rules

To store your validation rules, simply create a new public property in the **Config\Validation** class with the name of your group. This element will hold an array with your validation rules. As shown earlier, the validation array will have this prototype:

```
<?php

namespace Config;

// ...

class Validation extends BaseConfig
{
    // ...

    public array $signup = [
        'username'    => 'required|max_length[30]',
        'password'    => 'required|max_length[255]',
        'pass_confirm' => 'required|max_length[255]|matches[password]',
        'email'       => 'required|max_length[254]|valid_email',
    ];

    // ...
}
```

You can specify the group to use when you call the **run()** method:

```
$validation->run($data, 'signup');
```

Rules for General Use

The following is a list of all the native rules that are available to use:

Rule	Parameter	Description	Example
alpha	No	Fails if field has anything other than alphabetic characters in ASCII.	
alpha_dash	No	Fails if field contains anything other than alphanumeric characters, underscores or dashes in ASCII.	
alpha_numeric	No	Fails if field contains anything other than alphanumeric characters in ASCII.	
alpha_numeric_punct	No	Fails if field contains anything other than alphanumeric, space, or this limited set of punctuation characters: ~ (tilde), ! (exclamation), # (number), \$ (dollar), % (percent), & (ampersand), * (asterisk), - (dash), _ (underscore), + (plus), = (equals), (vertical bar), : (colon), . (period).	



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Rules for General Use

The following is a list of all the native rules that are available to use:

Rule	Parameter	Description	Example
alpha_numeric_space	No	Fails if field contains anything other than alphanumeric or space characters in ASCII.	
alpha_space	No	Fails if field contains anything other than alphabetic characters or spaces in ASCII.	
decimal	No	Fails if field contains anything other than a decimal number. Also accepts a + or - sign for the number.	
differs	Yes	Fails if field does not differ from the one in the parameter.	differs[field_name]

Rules for General Use

The following is a list of all the native rules that are available to use:

Rule	Parameter	Description	Example
exact_length	Yes	Fails if field length is not exactly the parameter value. One or more comma-separated values are possible.	exact_length[5] or exact_length[5,8,12]
field_exists	Yes	Fails if field does not exist. (This rule was added in v4.5.0.)	
greater_than	Yes	Fails if field is less than or equal to the parameter value or not numeric.	greater_than[8]
greater_than_equal_to	Yes	Fails if field is less than the parameter value, or not numeric.	greater_than_equal_to[5]



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Rules for General Use

The following is a list of all the native rules that are available to use:

Rule	Parameter	Description	Example
hex	No	Fails if field contains anything other than hexadecimal characters.	
if_exist	No	If this rule is present, validation will check the field only when the field key exists in the data to validate.	
in_list	Yes	Fails if field is not within a predetermined list.	in_list[red,blue,green]
integer	No	Fails if field contains anything other than an integer.	



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Rules for General Use

The following is a list of all the native rules that are available to use:

Rule	Parameter	Description	Example
is_natural	No	Fails if field contains anything other than a natural number: 0, 1, 2, 3, etc.	
is_natural_no_zero	No	Fails if field contains anything other than a natural number, except zero: 1, 2, 3, etc.	
is_not_unique	Yes	Checks the database to see if the given value exists. Can ignore records by field/value to filter (currently accept only one filter).	is_not_unique[table.field,where_field,where_value]
is_unique	Yes	Checks if this field value exists in the database. Optionally set a column and value to ignore, useful when updating records to ignore itself.	is_unique[table.field,ignore_field,ignore_value]



Rules for General Use

The following is a list of all the native rules that are available to use:

Rule	Parameter	Description	Example
less_than	Yes	Fails if field is greater than or equal to the parameter value or not numeric.	less_than[8]
less_than_equal_to	Yes	Fails if field is greater than the parameter value or not numeric.	less_than_equal_to[8]
matches	Yes	The value must match the value of the field in the parameter.	matches[field]
max_length	Yes	Fails if field is longer than the parameter value.	max_length[8]



Rules for General Use

The following is a list of all the native rules that are available to use:

Rule	Parameter	Description	Example
min_length	Yes	Fails if field is shorter than the parameter value.	min_length[3]
not_in_list	Yes	Fails if field is within a predetermined list.	not_in_list[red,blue,green]
numeric	No	Fails if field contains anything other than numeric characters.	
permit_empty	No	Allows the field to receive an empty array, empty string, null or false.	



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Rules for General Use

The following is a list of all the native rules that are available to use:

Rule	Parameter	Description	Example
regex_match	Yes	Fails if field does not match the regular expression.	regex_match[/regex/]
required	No	Fails if the field is an empty array, empty string, null or false.	
required_with	Yes	The field is required when any of the other fields is not empty() in the data.	required_with[field1,field2]
required_without	Yes	The field is required when any of the other fields is empty() in the data.	required_without[field1,field2]



Rules for General Use

The following is a list of all the native rules that are available to use:

Rule	Parameter	Description	Example
string	No	A generic alternative to the alpha* rules that confirms the element is a string	
timezone	No	Fails if field does not match a timezone per timezone_identifiers_list()	
valid_base64	No	Fails if field contains anything other than valid Base64 characters.	

Rules for General Use

The following is a list of all the native rules that are available to use:

Rule	Parameter	Description	Example
valid_cc_number	Yes	Verifies that the credit card number matches the format used by the specified provider. Current supported providers are: American Express (amex), China Unionpay (unionpay), Diners Club CarteBlanche (carteblanche), Diners Club (dinersclub), Discover Card (discover), Interpayment (interpayment), JCB (jcb), Maestro (maestro), Dankort (dankort), NSPK MIR (mir), Troy (troy), MasterCard (mastercard), Visa (visa), UATP (uatp), Verve (verve), CIBC Convenience Card (cibc), Royal Bank of Canada Client Card (rbc), TD Canada Trust Access Card (tdtrust), Scotiabank Scotia Card (scotia), BMO ABM Card (bmoabm), HSBC Canada Card (hsbc)	valid_cc_number(amex)

Rules for General Use

The following is a list of all the native rules that are available to use:

Rule	Parameter	Description	Example
valid_date	Yes	Fails if field does not contain a valid date. Any string that strtotime() accepts is valid if you don't specify an optional parameter that matches a date format. So it is usually necessary to specify the parameter.	valid_date[d/m/Y]
valid_email	No	Fails if field does not contain a valid email address.	
valid_emails	No	Fails if any value provided in a comma separated list is not a valid email.	



Rules for General Use

The following is a list of all the native rules that are available to use:

Rule	Parameter	Description	Example
valid_ip	Yes	Fails if the supplied IP is not valid. Accepts an optional parameter of ipv4 or ipv6 to specify an IP format.	valid_ip[ipv6]
valid_json	No	Fails if field does not contain a valid JSON string.	
valid_url	No	Fails if field does not contain (loosely) a URL. Includes simple strings that could be hostnames, like "codeigniter". Normally, valid_url_strict should be used.	
valid_url_strict	Yes	Fails if field does not contain a valid URL. You can optionally specify a list of valid schemas. If not specified, http,https are valid. This rule uses PHP's FILTER_VALIDATE_URL.	valid_url_strict[https]



